

## Deliverable 2

Generative Artificial Intelligence (580694), Spring 2026

**Due:** September 30, 2026, 23:59

**Weight:** 20% of the semester project

### 1. Objective

Deliverable 1 defined a task and diagnosed why a small open-weight model fails at it when prompted directly. Deliverable 2 turns that diagnosis into a running system. Your team commits to one of the three candidate models, implements a first solution against the diagnosed failure, and demonstrates it working on real inputs.

### 2. Requirements

**Model commitment.** Choose one of the three candidates proposed in Deliverable 1 and state why it was selected over the other two. You can include comparisons among them. If the team now uses a model that was not on that list, the change must be declared and defended on task grounds.

**First solution.** Implement one intervention against the diagnosed failure. Prompt structure, decomposition into steps, retrieval, tool use, constrained decoding, fine-tuning, or a combination. The choice is yours, but the link between the intervention and the specific failure diagnosed in Deliverable 1 must be explicit.

**Baseline comparison.** The solution must be shown against the direct prompting baseline on the same inputs. Without a baseline the improvement is not observable, and what cannot be observed cannot be graded.

**Working system.** Mockups, wireframes, and hardcoded outputs do not qualify. The pipeline must run end-to-end on the hardware declared in Deliverable 1.

**Failure case.** Identify at least one input where the solution still breaks or degrades, and show it.

### 3. Format and submission

**Video, maximum three minutes.** A screen recording of the system running. Only the first 3:00 are watched. Submit a link with open access.

**Technical document.** A vertical, single-page PDF compiled from LaTeX. Multi-page submissions and PDFs not produced in LaTeX will be rejected. The page must contain a pipeline diagram; the

selected model and its version; alternative strategies evaluated; measured results against the baseline over a set of inputs whose size is stated; and the known limits of the solution.

**Repository.** The link works, the code runs from the repository, and the README explains how to reproduce what the video shows.

#### **4. Evaluation rubric**

**Continuity with Deliverable 1 (10 points).** The system develops the same task and the same failure specified in Deliverable 1, under the same definition of a correct output. Deviations are declared and argued. A demonstration that quietly solves an easier problem than the one specified receives zero on this criterion, however well it runs.

**Demonstrated execution (10 points).** The video shows the pipeline running end to end on an input that was not chosen to flatter it, with the baseline visible on the same input. Editing that conceals the execution, or outputs that cannot be traced back to the repository, receive zero.

**Evidence of improvement (10 points).** Results are reported over a stated set of inputs, applying the same correctness criterion to the baseline and to the solution. One favorable example is an illustration, not evidence.

**Reading of the limits (10 points).** The failure case is real, and the team explains why it fails, not merely that it fails. Generic limitations such as more data or more time receive no credit.

**Model economy (10 points).** Full marks go to the team running the smallest model, measured in parameters. Teams running larger models are scored down relative to that reference. Size is credited only when the system meets the other criteria; a small model that does not run scores zero here.

**Repository and reproducibility (10 points).** The link works at the time of grading, the code runs from the repository, and the README allows a reader to understand what the video shows. A repository that only stores files, without instructions that lead to the demonstrated result, receives no credit.